

Date: 31/04/24

Exp. No. 02

EXP. TITLE: Write a program to implement the system call using `fork()`.

3

→ Algorithm:-

Step1:- Start the program.

Step2:- Declare variable `Pid` and `child id`.

Step3:- Get the `child id` value using system call `fork()`.

Step4:- If `child id` value is greater than zero then print as "I am parent process".

Step5:- If `child id != 0` then using `getpid()` system call process `Pid`.

Step6:- print "I am in parent process" and print process `Pid`.

Step7:- If `child id != 0` then using `getpid()` system call get parent process.

Step8:- print "I am in parent process" and print parent process `id`.

Step9:- Else `id` `child` value is less zero "I am child process".

Step10:- If `child id != 0` then using `getpid()` system call get the process `id`.

Step 1:- print "I am child process" and print process id

Step 2:- IF child pid $\neq$ 0 then using getppid() System call get the parent process id

Step 3:- print "I am in child process" and print process id

Step 4:- Stop ~~the~~ program.

Source code.

```
#* fork System call
```

```
# include <stdio.h>
# include <unistd.h>
# include <sys/types.h>
int main()
```

```
{
    int Pd, child id
    Pd = get pid()
    if (child Pd = fork()) > 0).
```

```
{
    printf("I am the parent process %d", id);
    printf("I am parent process %d",
           get pid());
    printf("I am in parent process in", get pid);
    else
    {
```

```
printf("I am in child process %d", id);
printf("I am in child process %d",
       get pid());
printf("I am in child process %d",
       get pid());
```

Anitha

exit()

⇒ Algorithm

Step 1:- Start the program.

Step 2:- Declare the variable p id & i as integer

Step 3:- Get the child id value using System call fork()

Step 4:- If child id where is less than zero. then print "fork failed"

Step 5:- Else if child id value is equal to zero if it is id value of child & then start child process execute & perform.

Step 6:- else perform Step 9.

Step 7:- use a for loop for almost five child process to be called

Step 8:- After execution of loop then print "child process ends"

Step 9:- Execute System call wait() make the parent to wait for child process get over.

once child process are terminated
the parent terminates have print
"parent process ends"

After both parent and child process
get terminated it execute wait()
system call permanently get deleted
from the operating system.

Stop the program.

⇒ program :

Source code:

```
#include <stdio.h>
#include <unistd.h>
int main()
{
    int p, pid;
    pid = fork();
    if (pid == 1)
    {
        printf("fork failed")
        exit(0);
    }
    else if (pid == 0)
    {
        printf("In child process starts");
        for (p=0; p<5; p++)
        {
            printf("In child process %d is called", p);
        }
        printf("In child process end");
    }
    else
    {
        wait(0);
        printf("In parent process end");
    }
    exit(0);
}
```

Anitha


```

#include <stdlib.h>
#include <errno.h>
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
int main()
{
    pid_t pid;
    int rv;
    switch (pid = fork())
    {

```

case -1:

```

        perror("fork");
        exit(1);
    }

```

case 0:

```

        printf("In CHILD: This is child process!\n");

```

```

        fflush(stdout);

```

```

        printf("CHILD: My PID is %d\n", getpid());

```

```

        printf("CHILD: Enter my exit status\n\n");

```

```

        scanf("%d", &rv);

```

```

        printf("CHILD: I'm outta here!\n");

```

```

        exit(rv);

```

```

    default:

```



```

printf("In PARENT: This parent process!\n");
printf("PARENT: My PID is %d\n",
      getppid());
fflush(stdout);
printf("PARENT: I'm now waiting
for my child to exit()...\n");
wait(&rv);
printf("PARENT: My child's PID is %d
      \n", ppid);
fflush(stdout);
printf("PARENT: My child's exit status
is: %d\n", WEXITSTATUS(rv));
printf("PARENT: I'm outta here!\n");
exit(0);
}
}

```

Another

Date: 15/10/24

Exp. No. 4

Exp. Title Write the program to
Implement system call
opendir(), readdir(), closedir()

11

ALGORITHM

Step1: Start the program.

Step2: In main function pass the argument

Step3: Create structure as stat buff and
variable as integer.

Step4: open ~~the~~ directory.

Step5: Read the Contents of directory (filename)

Step6: Display the Content of directory.

Step7: close ~~the~~ directory.

Step8: Stop ~~the~~ program.


```
#include <stdio.h>
#include <stdlib.h>
#include <dirent.h>
#include <errno.h>
#include <string.h>
```

```
void read_file(const char* filepath) {
    FILE* file = fopen(filepath, "r");
    if (file == NULL) {
        perror("Failed to open file");
        return;
    }
```

```
    char line[256];
    while (fgets(line, sizeof(line), file) !=
           NULL) {
        printf("%s", line);
```

```
    }
    fclose(file);
}
```

```
int main(int argc, char* argv[]) {
    if (argc != 2) {
        printf("Usage: %s <directory\n> - name\n", argv[0]);
        return EXIT_FAILURE;
    }
```

```
    struct dirent* dirent p;
    while ((dirent p = readdir(diro)) !=
           NULL) {
```

```
        if (strcmp(dirent p->d_name, ".") !=
            0 && strcmp(dirent p->d_name, "..") != 0
```



```

{
printf("file: %s\n", direntp->d_name);
if (direntp->d_type == DT_REG)
    char filepath[1024];
    snprintf(filepath, sizeof(filepath), "%s/%s",
        argv[1], direntp->d_name);
    printf("Contents of %s: \n", filepath);
    read_file(filepath);
}

printf("\n");
}

if (closed(drip) == -1)
{
    perror("Failed to close directory");
    return EXIT_FAILURE;
}

return EXIT_SUCCESS;
}

Anisha

```



```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
```

```
int main()
```

```
{
```

```
    int pid[15];
```

```
    int bt[15];
```

```
    int n;
```

```
    printf("Enter the number of processes");
```

```
    scanf("%d", &n);
```

```
    printf("Enter process id of all the processes");
```

```
    for (int i = 0; i < n; i++)
```

```
    {
```

```
        scanf("%d", &pid[i]);
```

```
    }
```

```
    printf("Enter burst time of all processes");
```

```
    for (int i = 0; i < n; i++)
```

```
    {
```

```
        int i, wt[n];
```

```
        wt[0] = 0;
```

```
        for (i = 1; i < n; i++)
```

```
        {
```

```
            wt[i] = bt[i-1] + wt[i-1];
```

```
        }
```

```
    printf("process ID Burst Time waiting Time Turn Around Time\n");
```



```
float twt = 0.0;
float tat = 0.0;
for (i = 0; i < n; i++)
```

```
{
    printf("%d\t\t", pid[i]);
    printf("%d\t\t", bt[i]);
    printf("%d\t\t", wt[i]);
    printf("%d\t\t", bt[i] + wt[i]);
    printf("\n");
```

```
    twt += wt[i];
    tat += wt[i] + bt[i];
}
```

```
float atl, awt;
```

```
awt = twt / n;
```

```
atl = tat / n;
```

```
printf("Avg. waiting time = %f\n", awt);
```

```
printf("Avg. turnaround time = %f", atl);
```

```
}
```

Anima